Self-Correcting Code Generation Using Small Language Models

Jeonghun Cho, Deokhyung Kang, Hyounghun Kim, Gary Lee
Natural Language Processing Group @ POSTECH
{jeonghuncho, deokhk, h.kim, gblee}@postech.ac.kr

EMNLP 2025
Suzhou, China | 中国苏州

Problem: Self-correction in code generation has relied on proprietary models.

To address this, we propose an RL-based method that enables open-source small LMs to perform self correction entirely **without external assistance.**

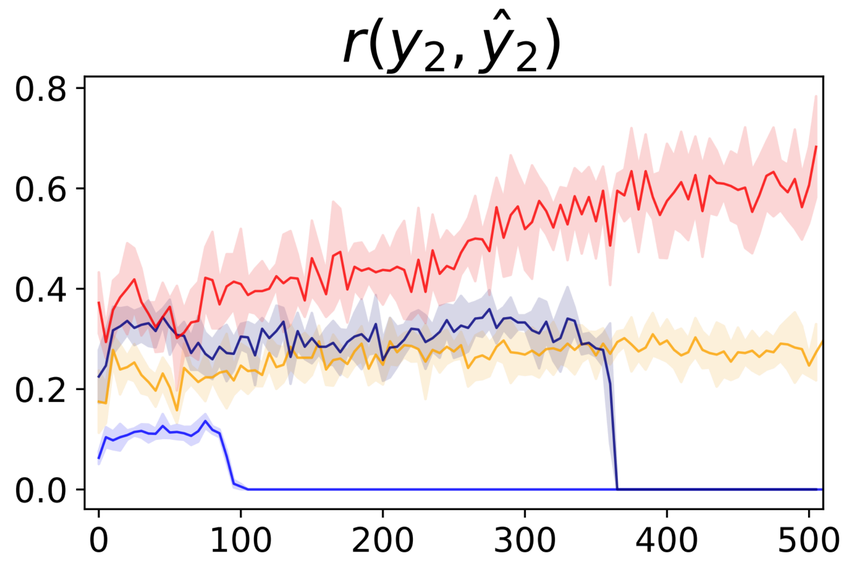
Observed Limitation

① Prompting-based Self-Correction

Method	Accuracy@t1	Accuracy@t2	$\Delta^{i\to c}$(t1, t2)	$\Delta^{c\to i}$(t1, t2)↓	Δ^{Acc}(t1, t2)
Simple-prompting	43.8%	44.6%	1.4%	0.6%	0.8%
Self-Refine	43.8%	38.2%	2.8%	8.4%	−5.6%
SETS	39.3%	38.8%	-	-	-

For small LMs (Qwen2.5-1.5B), we find that using very simple prompting (i.e., "fix it if something is wrong") outperforms the complex prompting strategies proposed in previous work.

② Training-based Self-Correction



Model: Qwen2.5-1.5B
Previous method: The only existing self-correction RL method

Red: RL training with the previous method, mimicking Gemini
Yellow: RL training without the previous method
Blue: RL training with the previous method

[Red] Qwen trained to mimic Gemini shows no issues when trained with the previous method.
[Yellow] Even without applying the method, the reward does not fully collapse.
[Blue] However, applying the same method directly to small language models fails to drive reward learning.

Multi-Turn MDP

$$\max_{\theta} \mathbb{E}_{(x,y)\sim\mathcal{D},\, \hat{y}\sim\pi_\theta}\Big[\sum_{t=1}^{T} r(y_t, \hat{y}_t) - \beta \mathbb{D}_{KL}\big(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_{ref}(\cdot \mid s_t)\big)\Big]$$

To enable multi-turn training, we first extend the single-turn MDP to a multi-turn formulation. By accumulating rewards at each turn, we apply a straightforward policy optimization procedure that supports training for up to T-turns.

Method (CoCoS)

① Accumulated reward function

$$R(\hat{y}_{1:T}) = \gamma^{T-1} r_1 + \sum_{t=2}^{T} \gamma^{T-t}(r_t - r_{t-1})$$

Unlike conventional RL, we use the discount factor to amplify the most recent turn.
Expected effect: The model performs corrections, subtracting the previous turn's reward to fix mistakes while preserving correct answers.

② Progressive reward

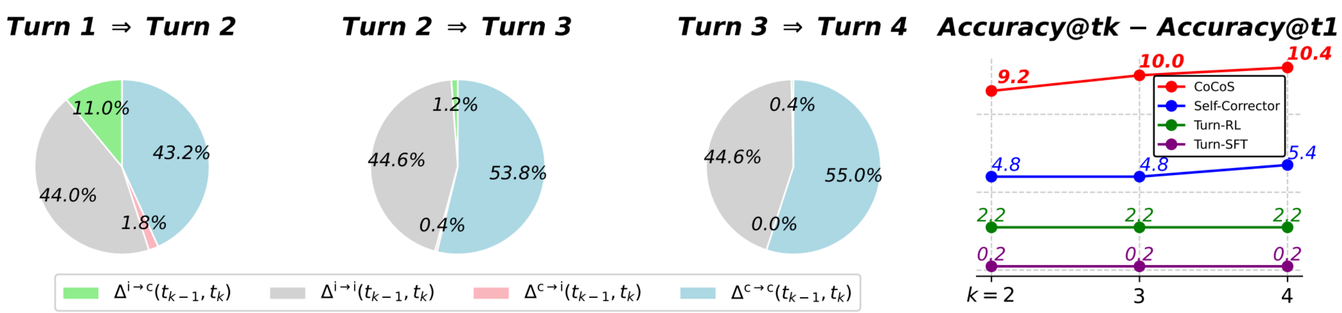
$$r_t = \frac{1}{K}\sum_{k=1}^{K} \mathbb{I}\{u_k(\hat{y}_t) \text{ passes}\}$$

Unlike the typical binary reward that only reflects whether all test cases pass, we assign rewards per individual test case.
Expected effect: Encourage the model to make incremental corrections.

$$\therefore \max_{\theta} \mathbb{E}_{(x,y)\sim\mathcal{D},\, \hat{y}\sim\pi_\theta}\Big[R(\hat{y}_{1:T}) - \beta \sum_{t=1}^{T} \mathbb{D}_{KL}\big(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_{ref}(\cdot \mid s_t)\big)\Big]$$

Result

Method	Accuracy@t1	Accuracy@t2	$\Delta^{i\to c}$(t1, t2)	$\Delta^{c\to i}$(t1, t2)↓	Δ^{Acc}(t1, t2)
Boost model	9.0%	10.2%	1.2%	0.0%	1.2%
Turn-SFT	44.2%	44.4%	1.4%	1.2%	0.2%
Self-Corrector	43.8%	48.8%	11.2%	6.2%	5.0%
ReVISE	34.6%	41.2%	11.3%	4.9%	6.4%
Turn-RL	**46.4%**	48.6%	4.0%	1.8%	2.2%
CoCoS	45.0%	**54.2%**	11.0%	1.8%	9.2%



Dataset	Accuracy@t1	Accuracy@t2	$\Delta^{i\to c}_{unit}$	$\Delta^{c\to i}_{unit}$ ↓
	Binary / Progressive			
MBPP	45.0 / 45.0	51.2 / **54.2**	10.5 / **11.9**	3.4 / **1.9**
HumanEval	24.4 / **29.3**	27.4 / **32.3**	**9.6** / 4.7	5.0 / **3.1**
ODEX	23.0 / 23.0	25.7 / **28.9**	9.8 / **34.2**	3.9 / **0.0**

🔥 **For more details, please refer to the paper** 🔥